



# Documento de Arquitectura de Software

## HAMPIQ

*Plataforma Nacional de Interoperabilidad del Historial Médico*

**Universidad Nacional Mayor de San Marcos**  
Facultad de Ingeniería de Sistemas e Informática

**Grupo: 5**

Versión 1.0 | 2026

## Tabla de Contenido

|                                                                  |    |
|------------------------------------------------------------------|----|
| Tabla de Contenido.....                                          | 2  |
| 1. Introducción .....                                            | 4  |
| 1.1 Propósito .....                                              | 4  |
| 1.2 Alcance .....                                                | 4  |
| 1.3 Definiciones, Siglas y Abreviaturas.....                     | 4  |
| 1.3.1 Definiciones .....                                         | 4  |
| 1.3.2 Acrónimos .....                                            | 5  |
| 1.4 Referencias .....                                            | 5  |
| 1.5 Vista Global .....                                           | 5  |
| 2. Representación Arquitectónica.....                            | 6  |
| 3. Metas y Restricciones Arquitectónicas .....                   | 6  |
| 3.1 Metas .....                                                  | 6  |
| 3.2 Restricciones.....                                           | 6  |
| 4. Vista de Casos de Uso.....                                    | 8  |
| 4.1 Descripción del Negocio .....                                | 8  |
| 4.2 Identificación de los Procesos del Negocio.....              | 8  |
| 4.3 Procesos de Negocio Relevantes para el Sistema .....         | 8  |
| 4.3.1 PN1: Compartición del historial mediante token médico..... | 8  |
| 4.3.2 PN2: Registro y actualización del historial clínico.....   | 8  |
| 4.3.3 PN3: Atención en modo emergencia .....                     | 9  |
| 4.4 Modelo de Dominio.....                                       | 9  |
| 4.5 Identificación de los Actores .....                          | 10 |
| 4.6 Casos de Uso Relevantes para la Arquitectura .....           | 10 |
| 4.7 Descripción de los Casos de Uso Relevantes .....             | 11 |
| 4.7.1 CU-01: Registrar usuario.....                              | 11 |
| 4.7.2 CU-02: Iniciar sesión .....                                | 12 |
| 4.7.3 CU-03: Generar token médico.....                           | 13 |
| 4.7.4 CU-04: Consultar historial mediante token .....            | 14 |
| 4.7.5 CU-05: Modo emergencia .....                               | 15 |
| 4.7.6 CU-06: Consultar auditoría.....                            | 16 |
| 5. Vista Lógica .....                                            | 18 |
| 5.1 Estilo Arquitectónico .....                                  | 18 |
| 5.2 Arquitectura Lógica de la Aplicación .....                   | 18 |
| 5.2.1 Visión General .....                                       | 18 |

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| 5.3 Identificación de las Clases de Diseño .....                      | 19 |
| 5.3.1 Diagramas de Secuencia — Subsistema Seguridad .....             | 19 |
| 5.3.2 Diagramas de Secuencia — Subsistema Compartición y Acceso ..... | 19 |
| 5.3.3 Diagramas de Secuencia — Subsistema Auditoría y Control .....   | 21 |
| 5.3.4 Diagrama de Subsistemas .....                                   | 21 |
| 5.3.5 Agrupación de Clases — Subsistema Seguridad .....               | 22 |
| 5.3.6 Agrupación de Clases — Subsistema Gestión de Historial .....    | 23 |
| 5.3.7 Agrupación de Clases — Subsistema Compartición y Acceso .....   | 24 |
| 5.3.8 Agrupación de Clases — Subsistema Auditoría y Control .....     | 25 |
| 6. Vista de Despliegue .....                                          | 27 |
| 6.1 Dispositivo Cliente .....                                         | 27 |
| 6.2 Servidor de Aplicaciones .....                                    | 27 |
| 6.3 Servidor de Datos .....                                           | 27 |
| 7. Vista de Implementación .....                                      | 28 |
| 7.1 Descripción .....                                                 | 28 |
| 7.2 Diagrama de Componentes .....                                     | 28 |
| 7.3 Organización de los Módulos .....                                 | 28 |
| 8. Vista de Datos .....                                               | 29 |
| 8.1 Modelo de Datos .....                                             | 29 |
| 8.2 Tipo de Base de Datos .....                                       | 29 |
| 8.3 Consideraciones de Integridad y Seguridad de los Datos .....      | 30 |

# 1. Introducción

El presente documento describe la arquitectura de software de Hampiq, una plataforma nacional orientada a centralizar y dar acceso interoperable al historial clínico del ciudadano peruano. Como artefacto del proceso de desarrollo, organiza el sistema en un conjunto de vistas arquitectónicas que, en conjunto, permiten comprender las decisiones de diseño y su justificación.

La estructura del documento sigue el modelo de vistas 4+1 adoptado por RUP (Rational Unified Process), abarcando la vista de casos de uso, la vista lógica, la vista de despliegue, la vista de implementación y la vista de datos.

## 1.1 Propósito

Este documento brinda una descripción detallada de la arquitectura de Hampiq a través de distintas vistas, cada una de las cuales ilustra un aspecto particular del software. Su finalidad es ofrecer al lector —desarrolladores, docentes y evaluadores del proyecto— una visión global y comprensible del diseño, sirviendo como referencia técnica para la implementación y la evolución futura del sistema.

## 1.2 Alcance

El documento profundiza principalmente en la vista de casos de uso y en la vista lógica, e incorpora los elementos más relevantes de las vistas de despliegue, implementación y datos. Sobre estas vistas se especifica la distribución de responsabilidades por capas y la organización de los subsistemas que componen la plataforma.

En esta versión, el alcance de implementación cubre un frontend funcional y un backend operativo desplegados en entorno local, dejando las integraciones con entidades externas (RENIEC, hospitales y farmacias) planificadas como evolución posterior.

## 1.3 Definiciones, Siglas y Abreviaturas

### 1.3.1 Definiciones

| Término                  | Definición                                                                                                                                     |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Actor                    | Usuario o sistema externo que participa de un caso de uso.                                                                                     |
| Arquitectura de software | Conjunto de decisiones estructurales que definen los componentes del sistema, sus relaciones y los principios que guían su diseño y evolución. |
| Caso de uso              | Secuencia de acciones que el sistema realiza y que produce un resultado de valor observable para un actor.                                     |
| Historia clínica         | Registro longitudinal de los eventos clínicos de un paciente.                                                                                  |
| Token médico             | Código temporal que autoriza a un médico el acceso controlado al historial de un paciente.                                                     |
| Modo emergencia          | Mecanismo de acceso rápido a un subconjunto vital de datos del paciente sin requerir sesión completa.                                          |
| Auditoría                | Registro inmutable de las acciones realizadas sobre la información del sistema.                                                                |

### 1.3.2 Acrónimos

| Acrónimo | Significado                                                                   |
|----------|-------------------------------------------------------------------------------|
| RUP      | Rational Unified Process (Proceso Unificado de Rational).                     |
| UML      | Unified Modeling Language (Lenguaje de Modelado Unificado).                   |
| API      | Application Programming Interface (Interfaz de Programación de Aplicaciones). |
| REST     | Representational State Transfer.                                              |
| RBAC     | Role-Based Access Control (Control de Acceso Basado en Roles).                |
| MFA      | Multi-Factor Authentication (Autenticación Multifactor).                      |
| QoS      | Quality of Service (Calidad de Servicio).                                     |
| TTL      | Time To Live (tiempo de vida de un dato en caché).                            |

### 1.4 Referencias

| Documento                                               | Versión | Fecha |
|---------------------------------------------------------|---------|-------|
| Especificación de Requisitos de Software (SRS) — Hampiq | 1.0     | 2026  |
| Documento Maestro de Análisis y Diseño — Hampiq         | 1.0     | 2026  |

### 1.5 Vista Global

El documento detalla la arquitectura del software a desarrollar, tomando como base la plantilla del artefacto Documento de la Arquitectura de Software del proceso RUP. Se presentan de manera clara los casos de uso con impacto arquitectónico, empleando un lenguaje sencillo y directo, y se describen los subsistemas que componen la plataforma Hampiq.

## 2. Representación Arquitectónica

---

Para Hampiq se ha escogido una arquitectura en capas (layered architecture) sobre un backend de servicios REST, en lugar del esquema cliente-servidor monolítico tradicional. La elección responde a la naturaleza del sistema: información clínica altamente relacional, con requisitos estrictos de seguridad, trazabilidad e integridad referencial, y con una hoja de ruta de interoperabilidad nacional que exige bajo acoplamiento entre módulos.

La plataforma se organiza en tres capas principales. La capa de presentación es una aplicación de página única (SPA) construida con React, TypeScript y Tailwind, responsable de la interacción con el usuario. La capa de aplicación, implementada con FastAPI, expone una API REST y estructura la lógica de negocio en una cadena de responsabilidades clara: routers que reciben las peticiones, servicios que aplican las reglas de negocio y repositorios que median el acceso a los datos. La capa de datos persiste la información en PostgreSQL y emplea Redis para el manejo de sesiones y, de forma destacada, para los tokens médicos temporales mediante expiración automática por TTL.

Esta separación permite que cada capa evolucione de forma independiente, facilita las pruebas y aísla las decisiones tecnológicas, sentando las bases para una eventual migración hacia microservicios sin reescribir la lógica de negocio.

## 3. Metas y Restricciones Arquitectónicas

---

### 3.1 Metas

La arquitectura busca establecer una estructura comprensible para la implementación y escalable para el desarrollo futuro de la plataforma. Las metas que la guían son:

- Garantizar la seguridad y confidencialidad de la información clínica en todo momento.
- Asegurar la trazabilidad completa: toda acción sobre los datos debe quedar auditada.
- Mantener la integridad del historial, evitando el borrado físico de información clínica.
- Favorecer un diseño modular y de bajo acoplamiento, preparado para la interoperabilidad nacional.
- Ofrecer tiempos de respuesta adecuados para consultas frecuentes y para el acceso de emergencia.

### 3.2 Restricciones

- El frontend se desarrollará con React, TypeScript y Tailwind, por su idoneidad para SPAs modernas y por la afinidad técnica del equipo.
- El backend se desarrollará en Python con FastAPI, aprovechando su validación de entrada con Pydantic y su documentación OpenAPI automática.
- La persistencia principal será PostgreSQL, por la naturaleza relacional y normativa de los datos clínicos; Redis se usará para sesiones y tokens temporales.
- En esta versión el sistema se despliega en entorno local; las integraciones con RENIEC, hospitales y farmacias quedan fuera del alcance inmediato.
- Toda la comunicación entre cliente y servidor se realizará sobre HTTPS/TLS.



## 4. Vista de Casos de Uso

### 4.1 Descripción del Negocio

Hampiq centraliza el historial clínico del ciudadano peruano y regula su acceso. El paciente es el propietario de sus datos y decide cuándo y con quién compartirlos. Cuando requiere atención, puede generar un token temporal que habilita a un médico a consultar su historial durante una ventana acotada de tiempo y un número limitado de usos; al expirar el token, el acceso se cierra automáticamente. Cada consulta queda registrada en una auditoría inmutable que el propio paciente puede revisar.

Los establecimientos de salud registran los eventos clínicos —consultas, diagnósticos, imágenes y resultados de laboratorio— que se incorporan a la cronología del historial sin sobrescribir ni eliminar información previa. En situaciones críticas, el personal de emergencia puede acceder, mediante el escaneo de un código QR, a un subconjunto vital de datos (grupo sanguíneo, alergias, enfermedades críticas y contacto de emergencia) sin exponer el historial completo.

### 4.2 Identificación de los Procesos del Negocio

Se identifican tres procesos de negocio relevantes para el sistema:

- PN1: Compartición del historial mediante token médico.
- PN2: Registro y actualización del historial clínico.
- PN3: Atención en modo emergencia.

### 4.3 Procesos de Negocio Relevantes para el Sistema

#### 4.3.1 PN1: Compartición del historial mediante token médico

Constituye la interacción central de Hampiq. El paciente genera un token temporal, lo configura (duración y número de usos) y lo comparte con el médico. El sistema valida el token, concede el acceso, registra la auditoría correspondiente y revoca o expira el acceso al cumplirse las condiciones.

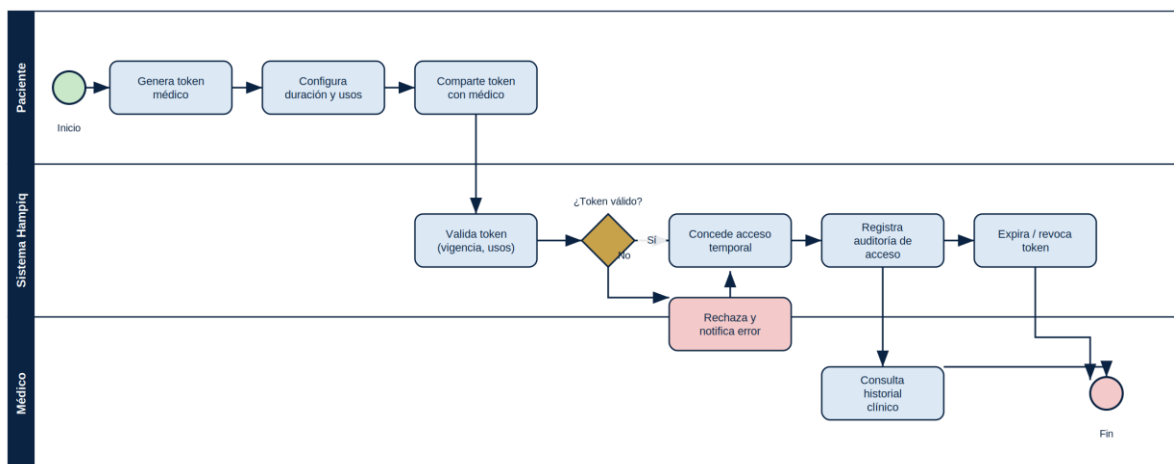


Figura 4.1 — Proceso de negocio PN1: Compartición del historial mediante token médico.

#### 4.3.2 PN2: Registro y actualización del historial clínico



El médico o establecimiento registra los eventos clínicos derivados de una atención. El sistema valida la información, almacena los estudios asociados, persiste el evento sin borrado físico, actualiza la cronología del historial y deja constancia en la auditoría. El paciente puede visualizar su historial actualizado.

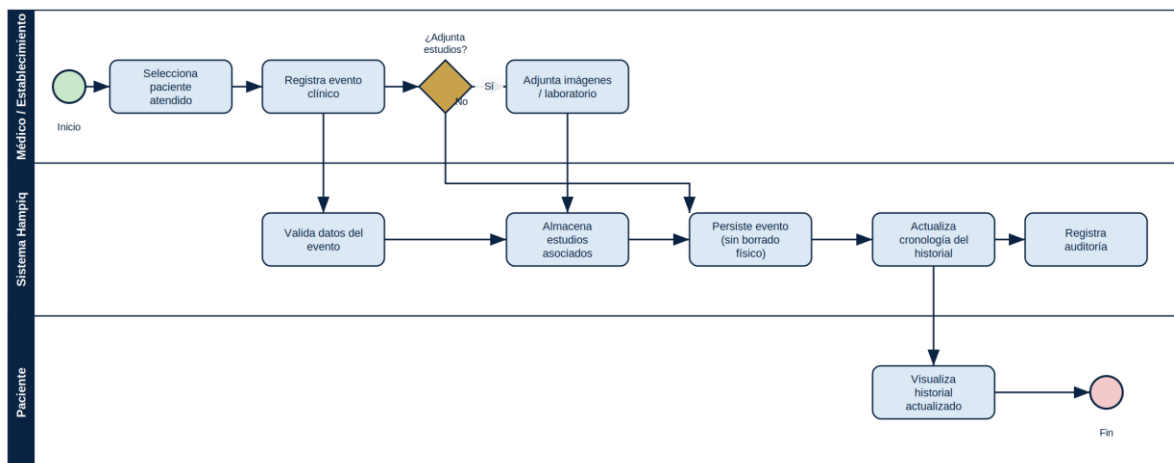


Figura 4.2 — Proceso de negocio PN2: Registro y actualización del historial clínico.

#### 4.3.3 PN3: Atención en modo emergencia

Ante una urgencia, el personal de emergencia escanea el código QR del paciente. El sistema valida el código y, de ser correcto, carga únicamente el subconjunto vital de datos —sin exponer el historial completo—, permitiendo una atención más rápida. Todo acceso de emergencia queda auditado.

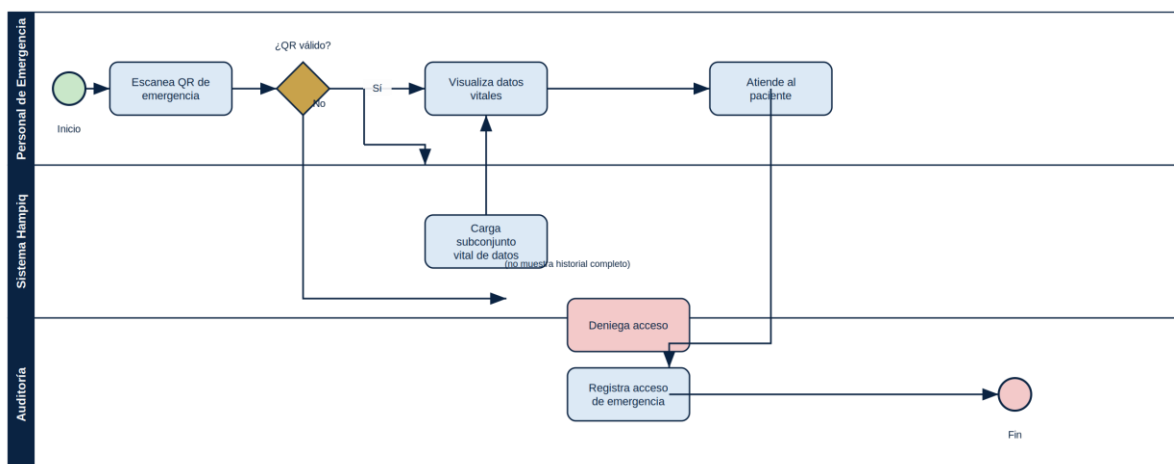


Figura 4.3 — Proceso de negocio PN3: Atención en modo emergencia.

### 4.4 Modelo de Dominio

El modelo de dominio identifica las entidades centrales de Hampiq y sus relaciones. El paciente posee un historial clínico que agrupa los eventos clínicos; cada evento puede incluir imágenes médicas y exámenes de laboratorio. El paciente genera tokens médicos que el médico utiliza para acceder al historial, y cada uso de token produce registros de auditoría supervisados por el administrador.

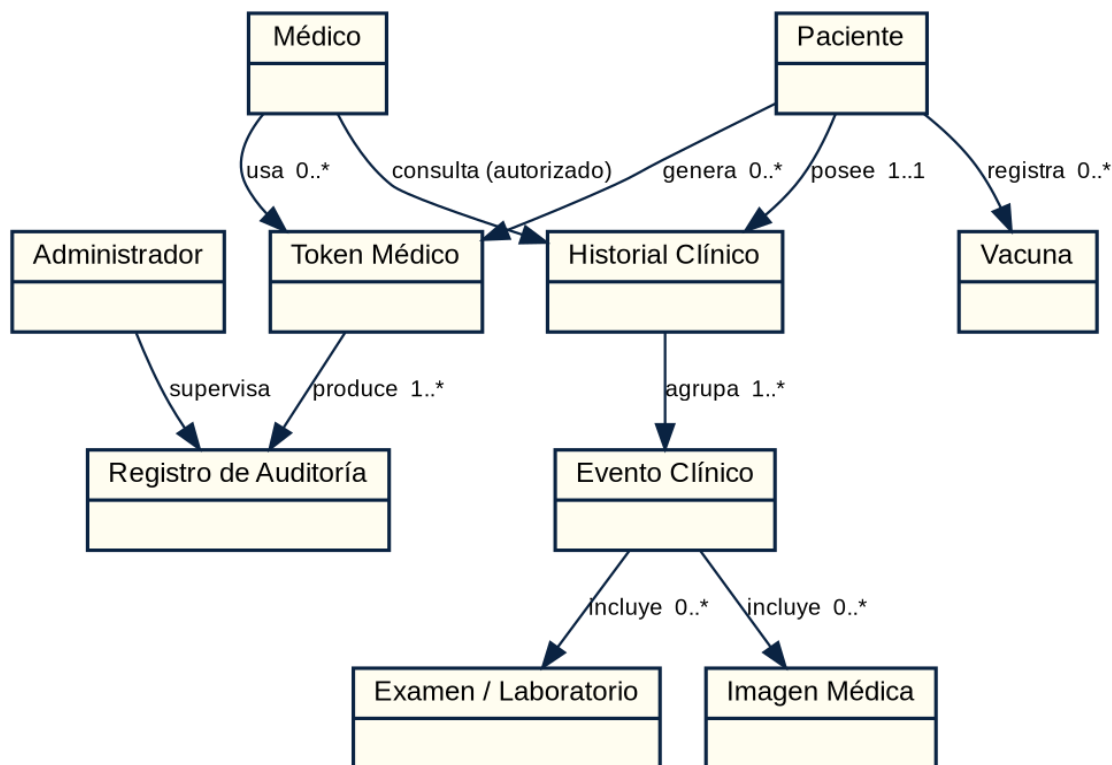


Figura 4.4 — Modelo de dominio de Hampiq.

## 4.5 Identificación de los Actores

Los actores que interactúan con el sistema son los siguientes:

- **Paciente:** propietario de sus datos clínicos. Gestiona su historial, genera tokens de acceso, consulta su auditoría y administra sus dispositivos.
- **Médico:** profesional de la salud que accede temporalmente al historial de un paciente mediante un token válido y registra eventos clínicos.
- **Administrador:** responsable de la gestión operativa, la supervisión y la auditoría del sistema.
- **Personal de emergencia:** actor que accede al subconjunto vital de datos mediante el modo emergencia.
- **Sistemas externos (RENIEC, hospitales, farmacias):** integraciones previstas como evolución futura para validación de identidad, sincronización de registros y comparación de precios de medicamentos.

## 4.6 Casos de Uso Relevantes para la Arquitectura

Los casos de uso con impacto arquitectónico se organizan en cuatro paquetes funcionales: Seguridad, Gestión de Historial, Compartición y Acceso, y Auditoría y Control. La siguiente figura presenta el paquete Negocio Principal con sus subpaquetes y los actores asociados.

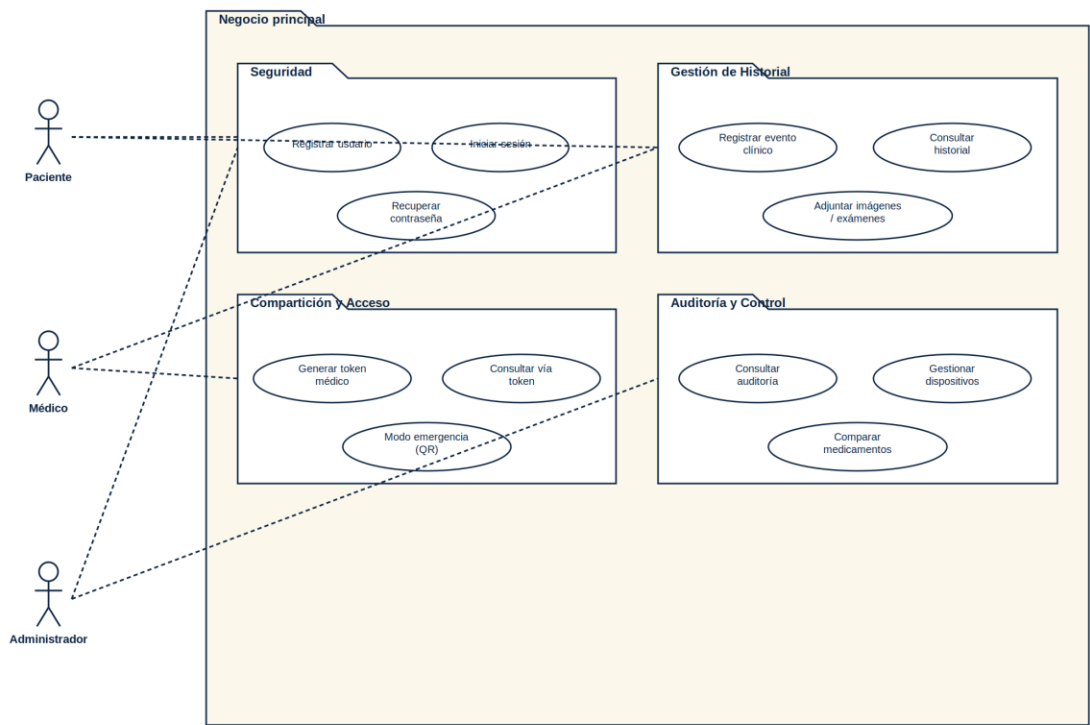


Figura 4.5 — Paquete Negocio Principal: casos de uso por subsistema y actores.

De este conjunto, los casos de uso desarrollados en detalle por su relevancia arquitectónica son: Registrar usuario, Iniciar sesión, Generar token médico, Consultar historial mediante token, Modo emergencia y Consultar auditoría.

4.7 Descripción de los Casos de Uso Relevantes

A continuación se describen en detalle los seis casos de uso con mayor relevancia arquitectónica de Hampiq. Para cada uno se presenta su ficha, sus flujos principal y alternos, y el wireframe de la interfaz asociada.

4.7.1 CU-01: Registrar usuario

|               |                                                                                                                    |
|---------------|--------------------------------------------------------------------------------------------------------------------|
| ID            | CU-01                                                                                                              |
| Caso de uso   | Registrar usuario                                                                                                  |
| Actor         | Paciente                                                                                                           |
| Descripción   | El paciente crea una cuenta a partir de su DNI, con el que el sistema autocompleta sus datos básicos de identidad. |
| Precondición  | Ninguna.                                                                                                           |
| Postcondición | Se ha registrado un nuevo usuario y este obtiene acceso al sistema.                                                |

Flujo principal

1. El paciente ingresa al sitio de Hampiq y selecciona la opción Crear cuenta.
2. El sistema muestra el formulario de registro con el campo DNI.
3. El paciente ingresa su DNI y presiona Validar.

4. El sistema verifica el formato del DNI y autocompleta nombres, apellidos y fecha de nacimiento.
5. El paciente completa correo, teléfono y una contraseña segura.
6. El paciente presiona Crear cuenta.
7. El sistema valida que el DNI no corresponda a una cuenta existente, registra al usuario y le concede acceso.

**Flujo alternativo 1: DNI con formato inválido (desde 3)**

1. El sistema indica que el DNI no tiene un formato válido.
2. El paciente corrige el dato e intenta nuevamente.

**Flujo alternativo 2: Usuario ya existente (desde 7)**

1. El sistema informa que ya existe una cuenta asociada al DNI.
2. No se registra un nuevo usuario y se sugiere iniciar sesión.

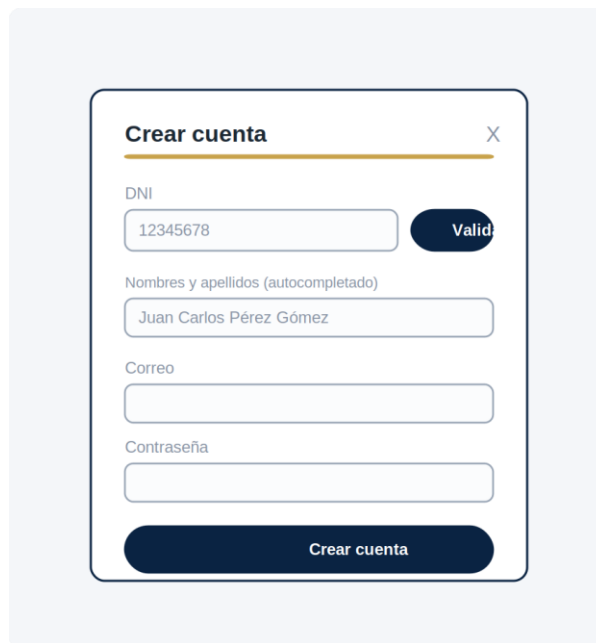


Figura 4.6 — Wireframe del registro de usuario (CU-01).

**4.7.2 CU-02: Iniciar sesión**

|                      |                                                                            |
|----------------------|----------------------------------------------------------------------------|
| <b>ID</b>            | CU-02                                                                      |
| <b>Caso de uso</b>   | Iniciar sesión                                                             |
| <b>Actor</b>         | Paciente / Médico / Administrador                                          |
| <b>Descripción</b>   | El usuario accede al sistema mediante sus credenciales (DNI y contraseña). |
| <b>Precondición</b>  | Haber registrado un usuario.                                               |
| <b>Postcondición</b> | El usuario accede al sistema con una sesión activa.                        |

**Flujo principal**

1. El usuario ingresa al sitio y presiona Iniciar sesión.
2. El sistema muestra el formulario con los campos DNI y contraseña.
3. El usuario ingresa sus credenciales y presiona Ingresar.

4. El sistema valida las credenciales y crea la sesión del usuario.
5. El usuario es redirigido a su panel principal según su rol.

**Flujo alternativo 1: Credenciales incorrectas (desde 3)**

1. El sistema muestra el mensaje de credenciales inválidas.
2. El usuario vuelve a intentar o cierra el formulario.

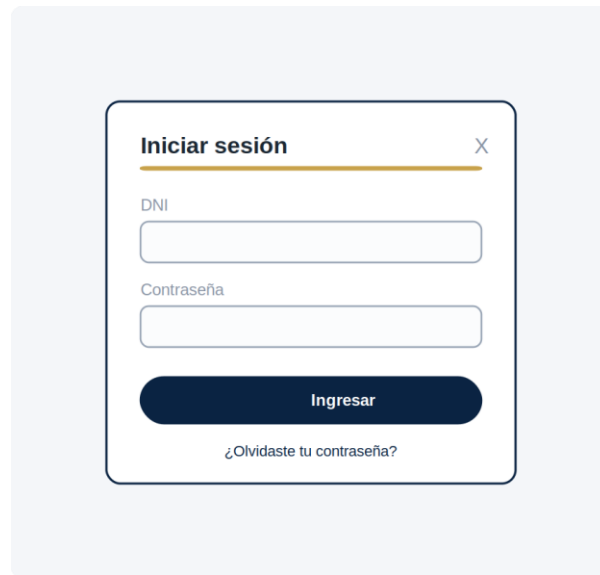


Figura 4.7 — Wireframe del inicio de sesión (CU-02).

**4.7.3 CU-03: Generar token médico**

|                      |                                                                                                              |
|----------------------|--------------------------------------------------------------------------------------------------------------|
| <b>ID</b>            | CU-03                                                                                                        |
| <b>Caso de uso</b>   | Generar token médico                                                                                         |
| <b>Actor</b>         | Paciente                                                                                                     |
| <b>Descripción</b>   | El paciente genera un código temporal que autoriza a un médico a consultar su historial de forma controlada. |
| <b>Precondición</b>  | El paciente ha iniciado sesión.                                                                              |
| <b>Postcondición</b> | Se genera un token con vigencia y número de usos definidos, listo para compartir.                            |

**Flujo principal**

1. El paciente accede a la opción Compartir historial.
2. El sistema muestra el formulario con los parámetros de duración y número de usos.
3. El paciente define la duración y el número de usos, y presiona Generar token.
4. El sistema crea el token, lo almacena con expiración por TTL y lo muestra en pantalla.
5. El paciente copia el token y lo comparte con el médico.

**Flujo alternativo 1: Revocación anticipada**

1. El paciente presiona Revocar antes del vencimiento.
2. El sistema invalida el token de inmediato y registra la acción en la auditoría.

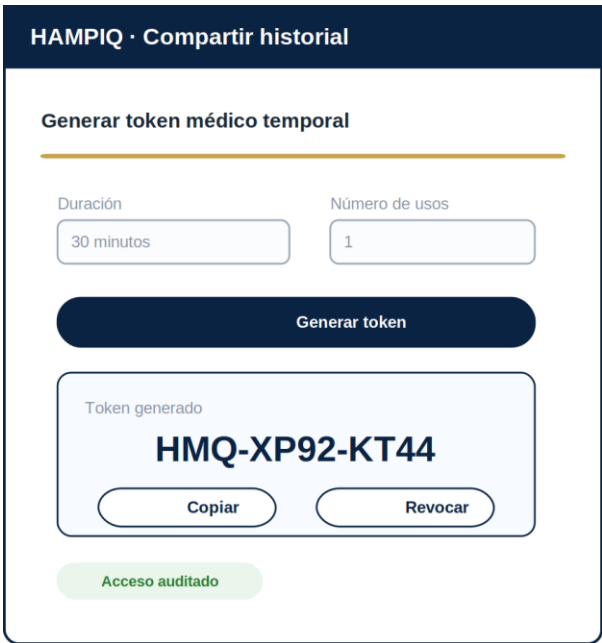


Figura 4.8 — Wireframe de generación de token médico (CU-03).

4.7.4 CU-04: Consultar historial mediante token

|               |                                                                                                   |
|---------------|---------------------------------------------------------------------------------------------------|
| ID            | CU-04                                                                                             |
| Caso de uso   | Consultar historial mediante token                                                                |
| Actor         | Médico                                                                                            |
| Descripción   | El médico accede temporalmente al historial clínico de un paciente introduciendo un token válido. |
| Precondición  | El paciente ha generado y compartido un token vigente.                                            |
| Postcondición | El médico visualiza el historial autorizado y el acceso queda registrado en la auditoría.         |

Flujo principal

1. El médico ingresa el token recibido en la pantalla de acceso.
2. El sistema valida la vigencia y el número de usos disponibles del token.
3. El sistema concede el acceso temporal y registra la auditoría correspondiente.
4. El médico visualiza la cronología del historial: eventos clínicos, imágenes y resultados.
5. Al cumplirse la duración o agotarse los usos, el sistema revoca el acceso automáticamente.

Flujo alternativo 1: Token inválido o expirado (desde 2)

1. El sistema rechaza el acceso e informa que el token no es válido.
2. El médico solicita al paciente un nuevo token.



Figura 4.9 — Wireframe de consulta de historial por token, vista del médico (CU-04).

#### 4.7.5 CU-05: Modo emergencia

|               |                                                                                                                                                       |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID            | CU-05                                                                                                                                                 |
| Caso de uso   | Modo emergencia                                                                                                                                       |
| Actor         | Personal de emergencia                                                                                                                                |
| Descripción   | El personal de emergencia accede a un subconjunto vital de datos del paciente mediante el escaneo de un código QR, sin exponer el historial completo. |
| Precondición  | El paciente cuenta con un código QR de emergencia.                                                                                                    |
| Postcondición | Se muestran los datos vitales y el acceso de emergencia queda auditado.                                                                               |

##### Flujo principal

1. El personal de emergencia escanea el código QR del paciente.
2. El sistema valida el código de emergencia.
3. El sistema carga únicamente el subconjunto vital: grupo sanguíneo, alergias, enfermedades críticas y contacto de emergencia.
4. El personal visualiza la información y procede con la atención.
5. El sistema registra el acceso de emergencia en la auditoría.

##### Flujo alternativo 1: Código QR inválido (desde 2)

1. El sistema deniega el acceso e informa que el código no es válido.
2. No se muestra ningún dato del paciente.

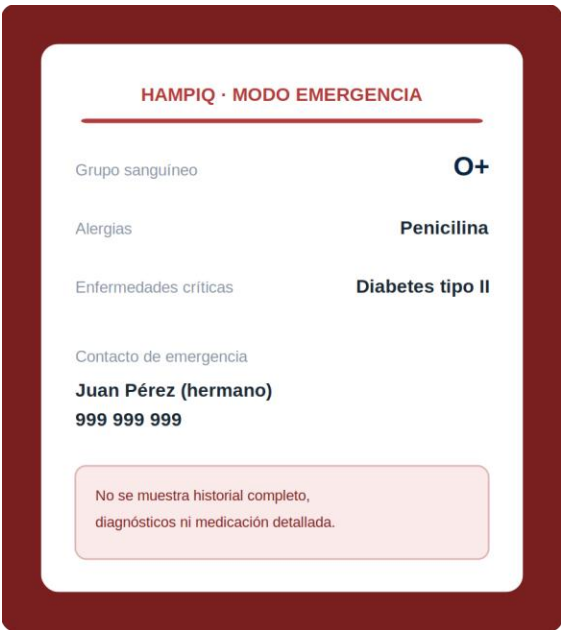


Figura 4.10 — Wireframe del modo emergencia (CU-05).

4.7.6 CU-06: Consultar auditoría

|               |                                                                                        |
|---------------|----------------------------------------------------------------------------------------|
| ID            | CU-06                                                                                  |
| Caso de uso   | Consultar auditoría                                                                    |
| Actor         | Paciente                                                                               |
| Descripción   | El paciente revisa el registro de todos los accesos realizados a su historial clínico. |
| Precondición  | El paciente ha iniciado sesión.                                                        |
| Postcondición | El paciente visualiza el historial de accesos y las sesiones activas.                  |

Flujo principal

1. El paciente accede a la sección Auditoría de accesos.
2. El sistema recupera y muestra los registros: fecha y hora, dispositivo, IP y acción.
3. El paciente puede revisar las sesiones activas y los dispositivos autorizados.
4. De ser necesario, el paciente puede cerrar sesiones o revocar dispositivos.

Flujo alternativo 1: Sin registros (desde 2)

1. El sistema indica que aún no existen accesos registrados.





Figura 4.11 — Wireframe de consulta de auditoría (CU-06).

## 5. Vista Lógica

La vista lógica describe la organización interna del sistema en términos de capas, subsistemas y clases de diseño. Presenta el estilo arquitectónico adoptado, la distribución de responsabilidades por capa y la realización de los casos de uso relevantes mediante diagramas de secuencia y de clases.

### 5.1 Estilo Arquitectónico

Hampiq adopta una arquitectura en capas sobre un backend de servicios REST. La capa de presentación se comunica con el backend exclusivamente a través de una API REST sobre HTTPS/TLS. La capa de aplicación organiza la lógica de negocio en una cadena clara de responsabilidades —routers, servicios y repositorios— que aísla las reglas del negocio del detalle de acceso a datos. La capa de datos persiste la información en PostgreSQL y emplea Redis para sesiones y tokens temporales.

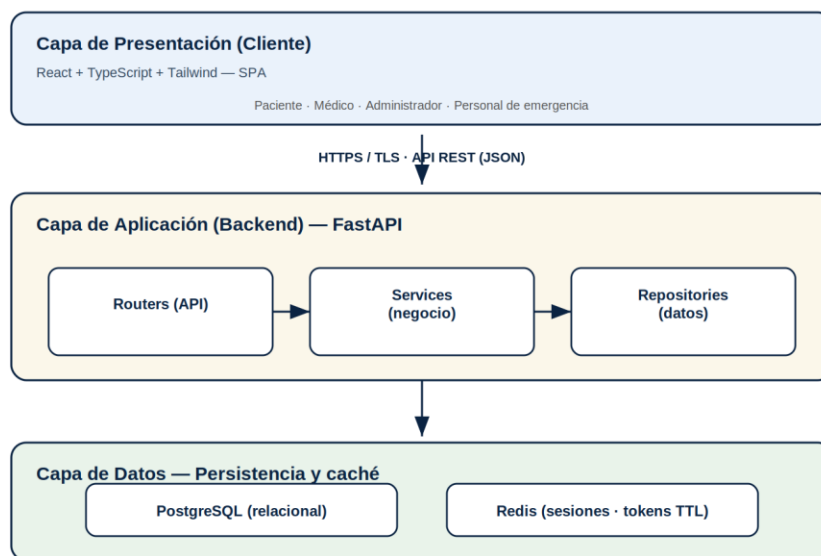


Figura 5.1 — Estilo arquitectónico en capas sobre backend REST.

### 5.2 Arquitectura Lógica de la Aplicación

#### 5.2.1 Visión General

La aplicación se estructura en cuatro subsistemas funcionales que agrupan las clases de diseño según la responsabilidad que cubren:

- Subsistema Seguridad: registro de usuarios, autenticación y gestión de sesiones.
- Subsistema Gestión de Historial: registro y consulta de eventos clínicos, imágenes y exámenes.
- Subsistema Compartición y Acceso: generación y validación de tokens médicos y modo emergencia.
- Subsistema Auditoría y Control: registro inmutable de accesos y gestión de dispositivos y sesiones.

### 5.3 Identificación de las Clases de Diseño

Para identificar las clases de diseño se realiza cada caso de uso relevante mediante un diagrama de secuencia, que muestra la colaboración entre las clases de interfaz (límite), los gestores (control) y las entidades persistentes (entidad).

#### 5.3.1 Diagramas de Secuencia — Subsistema Seguridad

CU-01: Registrar usuario. El paciente valida su DNI, el sistema autocompleta sus datos y el gestor de registro crea la entidad Usuario.

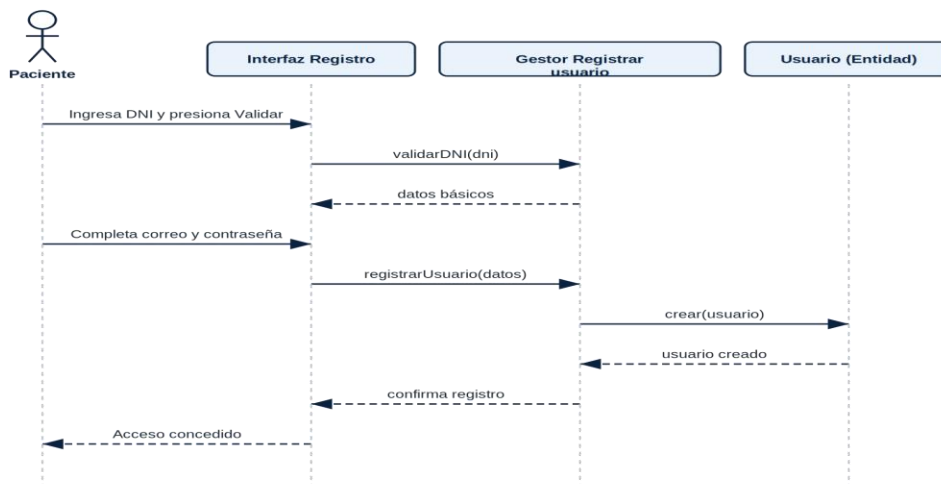


Figura 5.2 — Diagrama de secuencia del CU-01: Registrar usuario.

CU-02: Iniciar sesión. El gestor de inicio de sesión verifica las credenciales contra la entidad Usuario y crea la sesión activa.

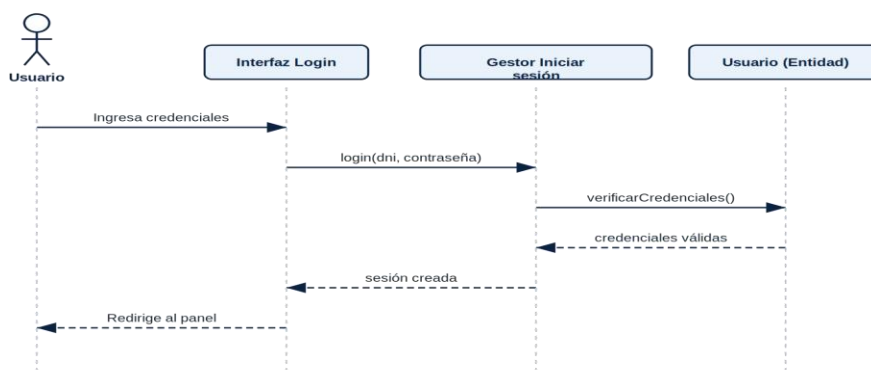


Figura 5.3 — Diagrama de secuencia del CU-02: Iniciar sesión.

#### 5.3.2 Diagramas de Secuencia — Subsistema Compartición y Acceso

CU-03: Generar token médico. El gestor de tokens crea el código, lo almacena en Redis con expiración por TTL y registra la generación en la auditoría.

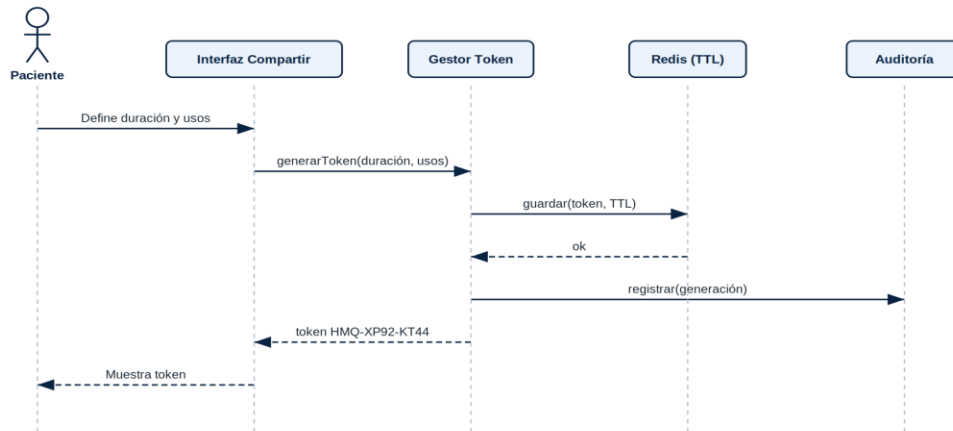


Figura 5.4 — Diagrama de secuencia del CU-03: Generar token médico.

CU-04: Consultar historial mediante token. El gestor de tokens valida la vigencia del código y, una vez autorizado, el gestor de historial recupera la cronología y registra el acceso.

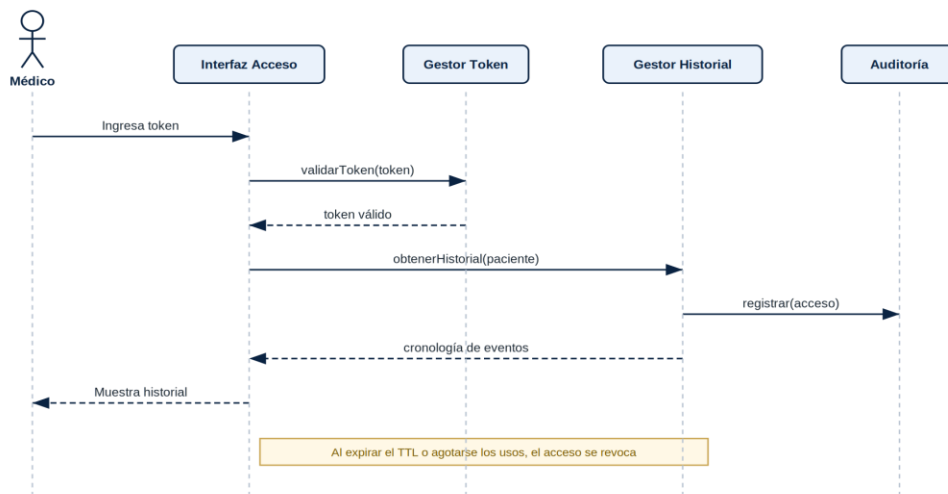


Figura 5.5 — Diagrama de secuencia del CU-04: Consultar historial mediante token.

CU-05: Modo emergencia. El gestor de emergencia valida el código QR, recupera únicamente el subconjunto vital de datos y registra el acceso de emergencia.

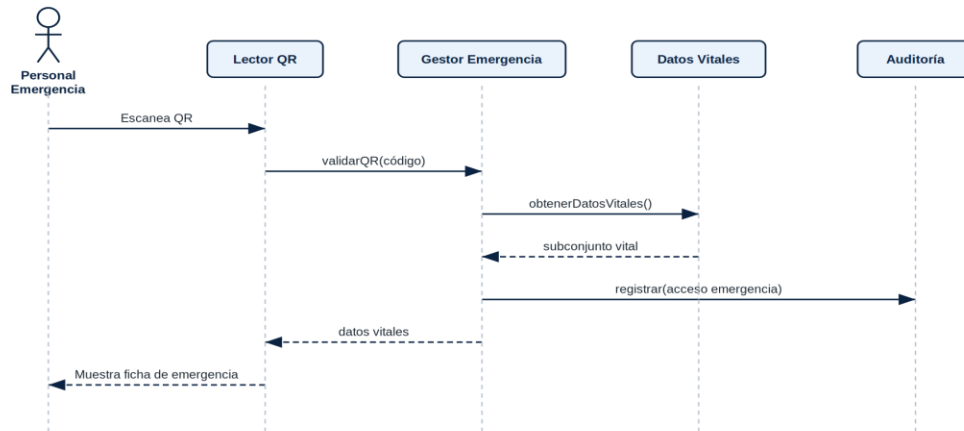


Figura 5.6 — Diagrama de secuencia del CU-05: Modo emergencia.

### 5.3.3 Diagramas de Secuencia — Subsistema Auditoría y Control

CU-06: Consultar auditoría. El gestor de auditoría recupera los registros de acceso del usuario y las sesiones activas para su visualización.

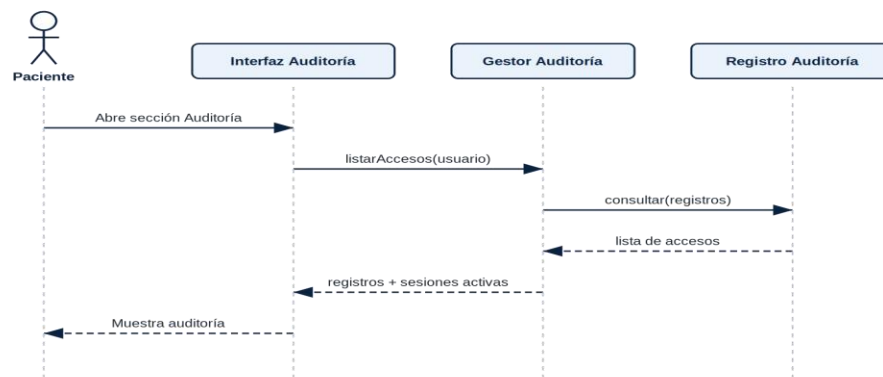


Figura 5.7 — Diagrama de secuencia del CU-06: Consultar auditoría.

### 5.3.4 Diagrama de Subsistemas

El siguiente diagrama presenta los cuatro subsistemas y sus dependencias. El subsistema Seguridad es transversal: el resto de subsistemas dependen de él para autenticar al usuario. Compartición y Acceso y Gestión de Historial colaboran para servir el historial autorizado, y ambos registran sus operaciones en Auditoría y Control.

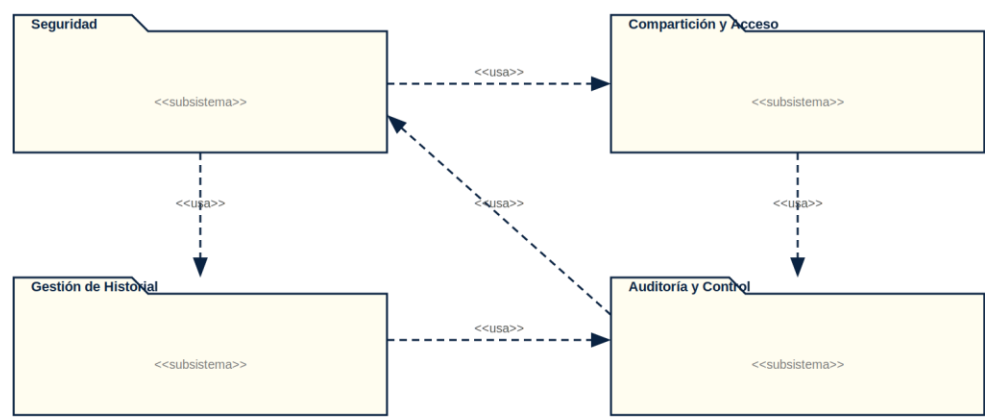


Figura 5.8 — Diagrama de subsistemas y sus dependencias.

5.3.5 Agrupación de Clases — Subsistema Seguridad

Clases: InterfazSitioPrincipal, InterfazLogin, InterfazRegistro, GestorIniciarSesion, GestorRegistrarUsuario y Usuario.

|                                                                                                      |
|------------------------------------------------------------------------------------------------------|
| <b>Clase: GestorIniciarSesion</b>                                                                    |
| <b>Responsabilidades</b><br>Autenticar las credenciales del usuario y generar y gestionar la sesión. |
| <b>Colaboraciones</b><br>Usuario                                                                     |

|                                                                                                             |
|-------------------------------------------------------------------------------------------------------------|
| <b>Clase: GestorRegistrarUsuario</b>                                                                        |
| <b>Responsabilidades</b><br>Validar el DNI, verificar la unicidad del usuario y almacenar el nuevo usuario. |
| <b>Colaboraciones</b><br>Usuario                                                                            |

|                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------|
| <b>Clase: Usuario</b>                                                                                             |
| <b>Responsabilidades</b><br>Contener los atributos del usuario y su estado de autenticación (contraseña cifrada). |
| <b>Colaboraciones</b><br>GestorIniciarSesion, GestorRegistrarUsuario                                              |

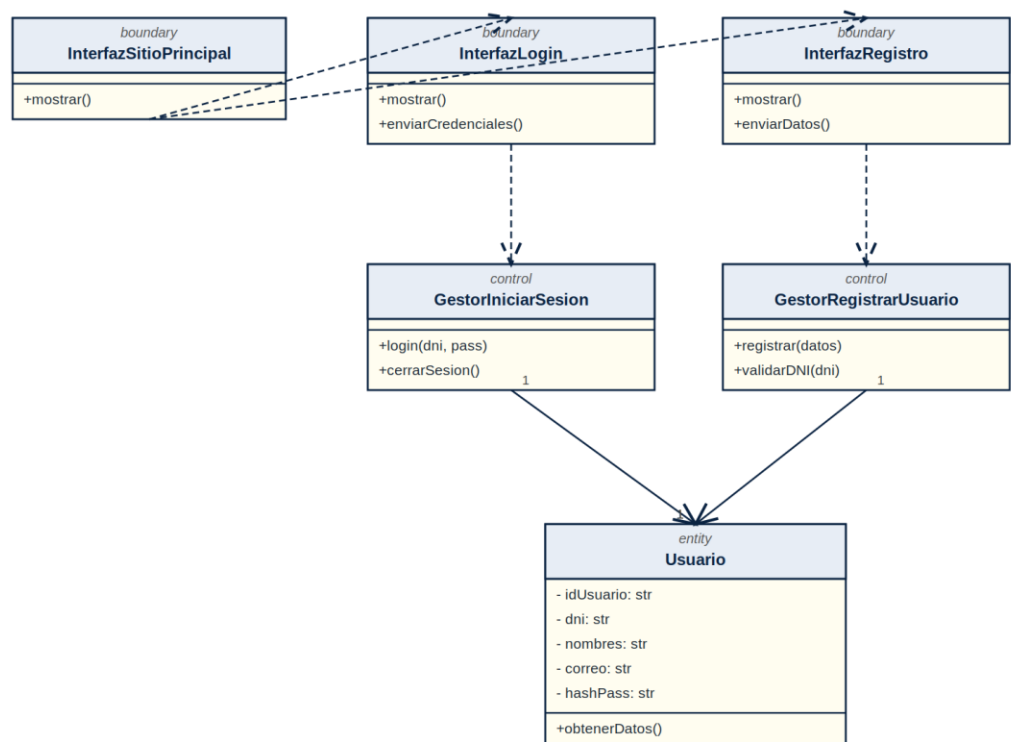


Figura 5.9 — Diagrama de clases de diseño del subsistema Seguridad.

5.3.6 Agrupación de Clases — Subsistema Gestión de Historial

Clases: InterfazHistorial, InterfazRegistroEvento, GestorHistorial, GestorRegistrarEvento, HistorialClinico, EventoClinico y Estudio.

|                                                                                                            |
|------------------------------------------------------------------------------------------------------------|
| <b>Clase: GestorHistorial</b>                                                                              |
| <b>Responsabilidades</b><br>Recuperar y ordenar cronológicamente los eventos del historial de un paciente. |
| <b>Colaboraciones</b><br>HistorialClinico, EventoClinico                                                   |

|                                                                                                              |
|--------------------------------------------------------------------------------------------------------------|
| <b>Clase: GestorRegistrarEvento</b>                                                                          |
| <b>Responsabilidades</b><br>Crear nuevos eventos clínicos y adjuntar estudios asociados, sin borrado físico. |
| <b>Colaboraciones</b><br>EventoClinico, Estudio                                                              |

|                                                                                                           |
|-----------------------------------------------------------------------------------------------------------|
| <b>Clase: EventoClinico</b>                                                                               |
| <b>Responsabilidades</b><br>Contener la información de un evento clínico (fecha, diagnóstico, severidad). |
| <b>Colaboraciones</b><br>HistorialClinico, Estudio                                                        |

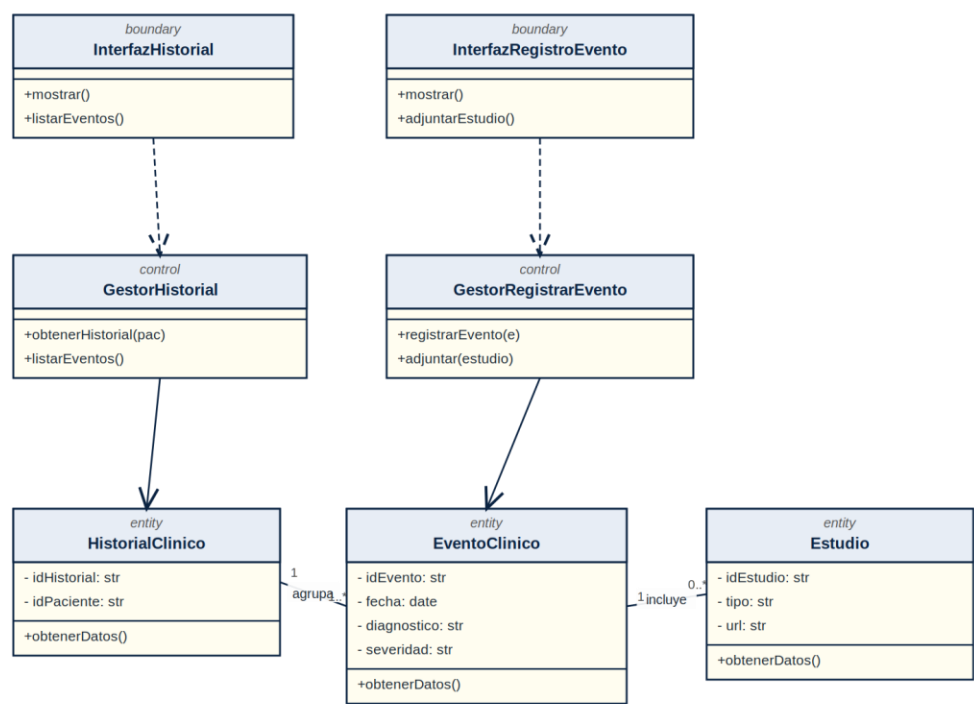


Figura 5.10 — Diagrama de clases de diseño del subsistema Gestión de Historial.

5.3.7 Agrupación de Clases — Subsistema Compartición y Acceso

Clases: InterfazCompartir, InterfazAccesoMedico, InterfazModoEmergencia, GestorToken, GestorEmergencia, TokenMedico y DatosVitales.

Clase: GestorToken

**Responsabilidades**  
Generar, validar y revocar tokens médicos, gestionando su expiración por TTL en Redis.

**Colaboraciones**  
TokenMedico

Clase: GestorEmergencia

**Responsabilidades**  
Validar el código QR y recuperar únicamente el subconjunto vital de datos del paciente.

**Colaboraciones**  
DatosVitales

Clase: TokenMedico

**Responsabilidades**  
Contener el código temporal, su vigencia y el número de usos disponibles.

**Colaboraciones**  
GestorToken



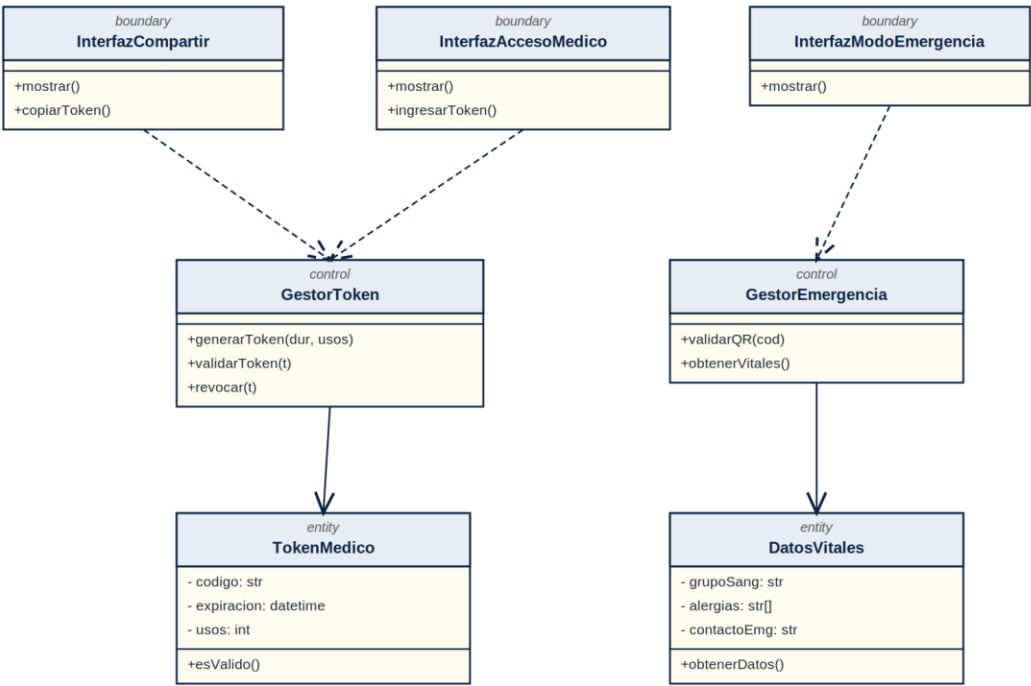


Figura 5.11 — Diagrama de clases de diseño del subsistema Compartición y Acceso.

5.3.8 Agrupación de Clases — Subsistema Auditoría y Control

Clases: InterfazAuditoria, InterfazDispositivos, GestorAuditoria, GestorDispositivos, RegistroAuditoria y Sesion.

Clase: GestorAuditoria

Responsabilidades

Registrar de forma inmutable cada acceso y recuperar el historial de accesos del usuario.

Colaboraciones

RegistroAuditoria

Clase: GestorDispositivos

Responsabilidades

Listar las sesiones activas del usuario y permitir la revocación de dispositivos.

Colaboraciones

Sesion

Clase: RegistroAuditoria

Responsabilidades

Contener los datos de un acceso: fecha y hora, IP, dispositivo y acción realizada.

Colaboraciones

GestorAuditoria

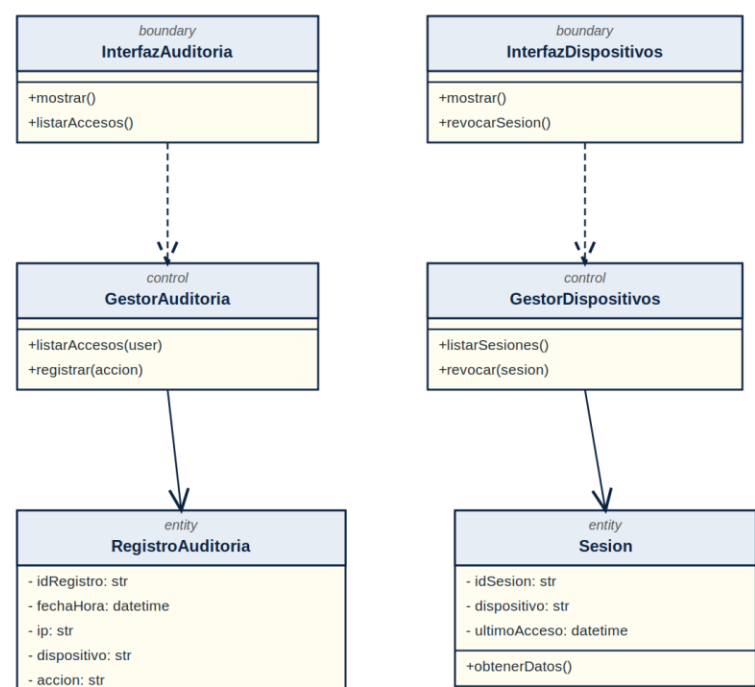


Figura 5.12 — Diagrama de clases de diseño del subsistema Auditoría y Control.

## 6. Vista de Despliegue

La vista de despliegue describe la distribución física de los componentes del sistema sobre los nodos de ejecución. En esta versión, los tres nodos se ejecutan en un entorno local; la topología está preparada para escalar a servidores independientes en fases posteriores.

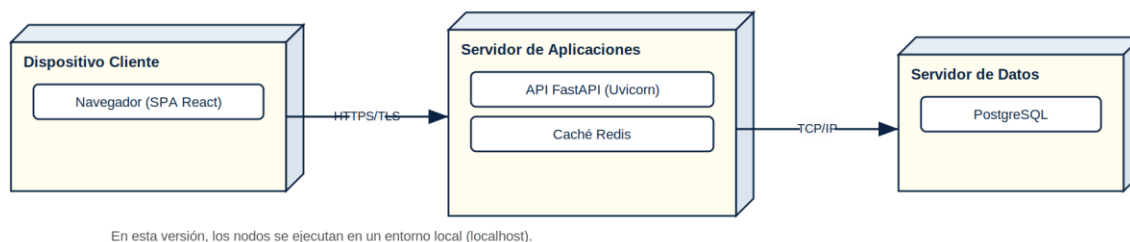


Figura 6.1 — Diagrama de despliegue de Hampiq (entorno local).

### 6.1 Dispositivo Cliente

Ejecuta el navegador web con la aplicación de página única (SPA) de React. Se comunica con el servidor de aplicaciones mediante HTTPS/TLS.

### 6.2 Servidor de Aplicaciones

Aloja la API REST construida con FastAPI sobre Uvicorn y la caché Redis empleada para sesiones y tokens temporales. Concentra la lógica de negocio y la aplicación de las políticas de seguridad.

- Procesador: 2 vCPU.
- Memoria RAM: 2 GB.
- Almacenamiento: 10 GB.

### 6.3 Servidor de Datos

Ejecuta el motor PostgreSQL, responsable de la persistencia de las entidades del sistema con integridad referencial. Se comunica con el servidor de aplicaciones mediante TCP/IP.

- Procesador: 2 vCPU.
- Memoria RAM: 2 GB.
- Almacenamiento: 20 GB.

## 7. Vista de Implementación

### 7.1 Descripción

La vista de implementación presenta el sistema en términos de componentes de software —los ficheros y módulos de código fuente— y su organización. Hampiq se estructura en un frontend, un backend y módulos transversales de infraestructura y seguridad.

### 7.2 Diagrama de Componentes

El frontend se organiza en módulos de páginas, componentes de interfaz, cliente de servicios y gestión de estado. El backend de FastAPI se estructura en routers, servicios, repositorios y esquemas de validación. Los módulos transversales cubren autenticación (JWT y RBAC) y trazabilidad de auditoría, y la infraestructura de datos integra PostgreSQL y Redis.

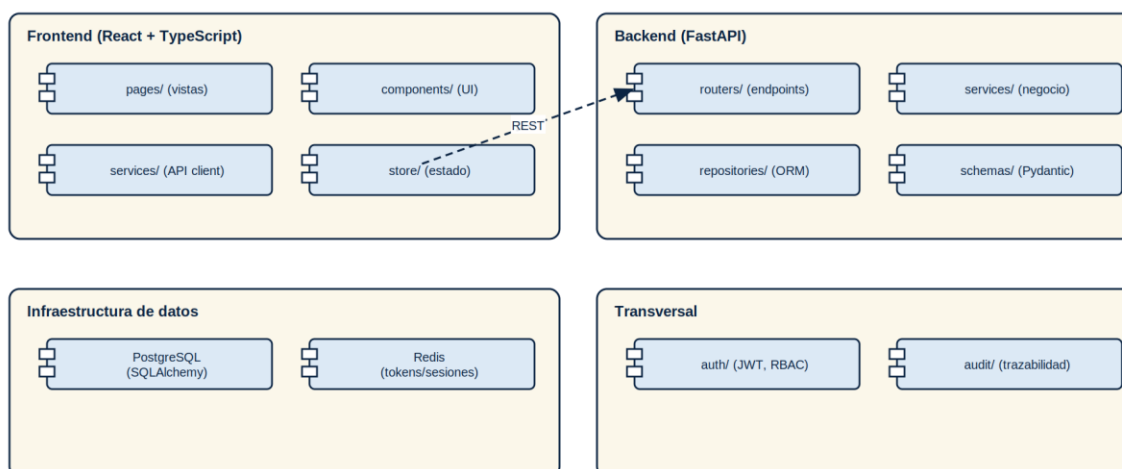


Figura 7.1 — Diagrama de componentes de Hampiq.

### 7.3 Organización de los Módulos

- Frontend (React + TypeScript): `pages/`, `components/`, `services/`, `store/`.
- Backend (FastAPI): `routers/`, `services/`, `repositories/`, `schemas/`.
- Transversal: `auth/` (JWT, RBAC), `audit/` (trazabilidad).
- Infraestructura de datos: PostgreSQL (SQLAlchemy) y Redis.

## 8. Vista de Datos

### 8.1 Modelo de Datos

El modelo de datos define las entidades persistentes de Hampiq y sus relaciones, con sus claves primarias (PK), foráneas (FK) y restricciones de unicidad (UQ). El diseño respeta la regla de negocio de no borrado físico mediante el atributo `deletedAt` en los eventos clínicos, que permite el archivado lógico preservando la trazabilidad.

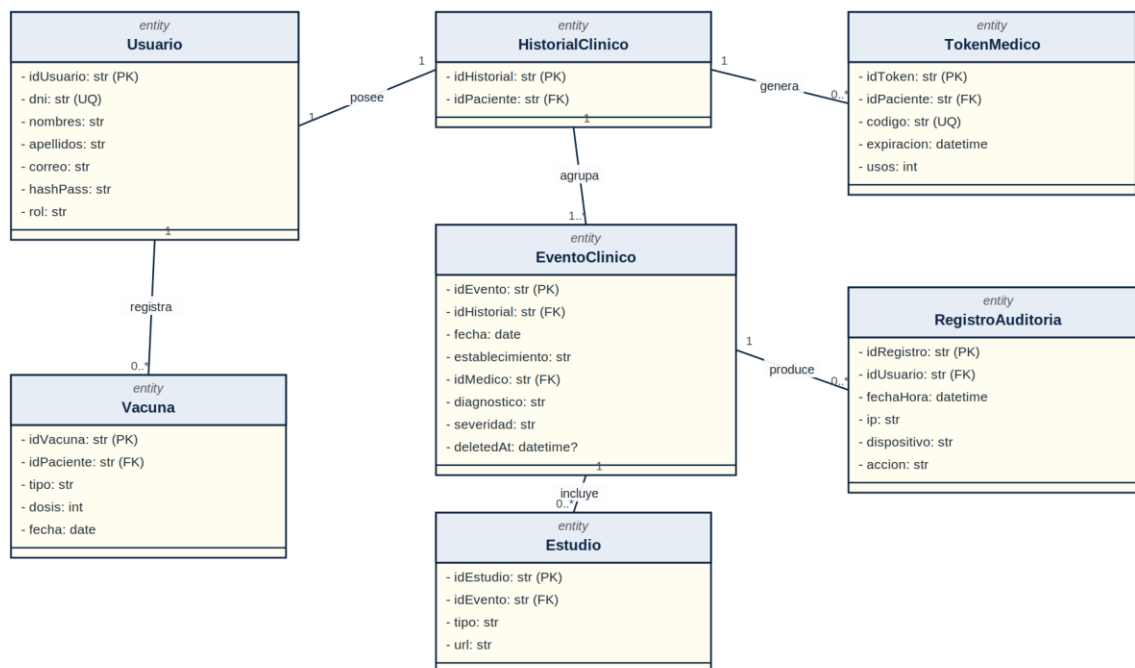
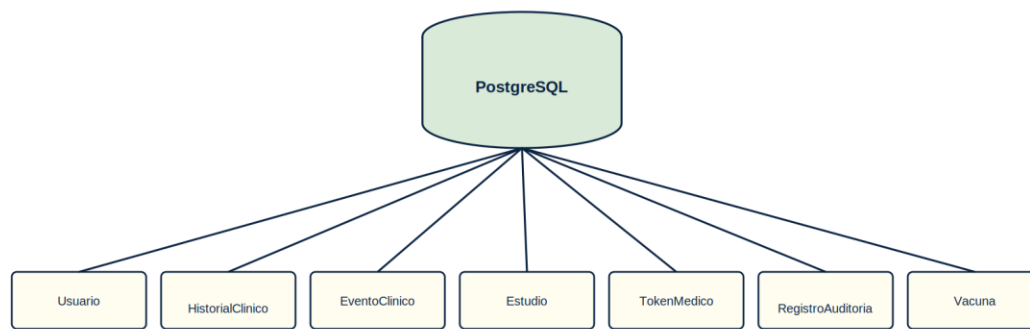


Figura 8.1 — Modelo de datos (entidades persistentes) de Hampiq.

### 8.2 Tipo de Base de Datos

Hampiq emplea una base de datos centralizada en PostgreSQL. La elección de un motor relacional responde a la naturaleza altamente estructurada y normativa de los datos clínicos: las transacciones ACID garantizan que operaciones como un acceso médico y su registro de auditoría se confirmen de forma atómica, y las claves foráneas aseguran la integridad referencial entre pacientes, historiales, eventos y estudios.



Base de datos centralizada (PostgreSQL) con integridad referencial y borrado lógico.

*Figura 8.2 — Base de datos centralizada (PostgreSQL).*

### 8.3 Consideraciones de Integridad y Seguridad de los Datos

- Integridad referencial garantizada mediante claves foráneas entre todas las entidades relacionadas.
- Borrado lógico (deletedAt) en lugar de borrado físico, para preservar el historial clínico íntegro.
- Contraseñas almacenadas mediante hash seguro; nunca en texto plano.
- Tokens médicos gestionados en Redis con expiración automática por TTL.
- Registro de auditoría inmutable para toda operación sobre la información clínica.